

Student No: ST10108083

Student Name: Kamogelo Fortune Mafuna

Module Name: Enterprise Application Development

GLOBAL LOGISTICS MANAGEMENT SYSTEM (GLMS)

Enterprise Analysis & Architecture Report

1. Executive Summary & Introduction

According to the background case study, TechMove Logistics currently relies on **fragmented legacy systems**, including **spreadsheets**, **emails**, and **manual communication channels**. This has led to operational inefficiencies such as lost invoices, inconsistent contract tracking, compliance risks, and delayed service delivery.

The proposed Global Logistics Management System (GLMS) is an enterprise-grade, web-based platform designed to centralize logistics operations, automate workflows, and ensure data consistency across the organization. The system introduces a microservices-based architecture, enabling scalability, high availability, and seamless integration with external systems such as currency exchange APIs.

From a strategic perspective, GLMS aligns with TechMove's long-term vision of digital transformation by improving operational efficiency, ensuring regulatory compliance, and supporting future expansion, including acquisitions and increased contract volumes. The architecture is designed not only to solve current inefficiencies but also to provide a scalable and resilient foundation for future growth.

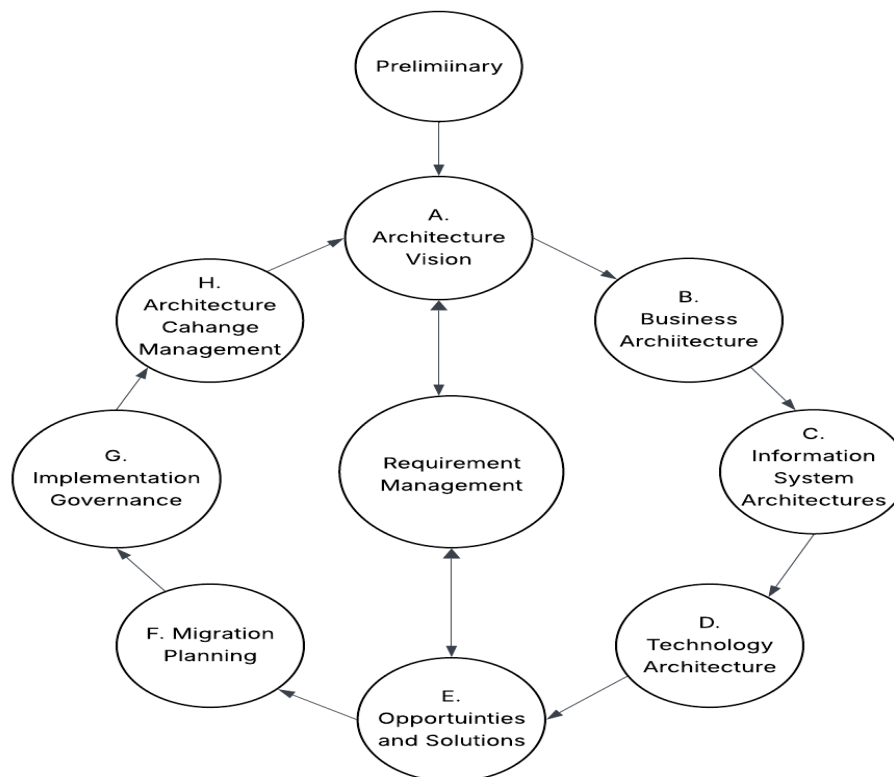
2. Enterprise Framework Strategy (TOGAF)

For this architecture of GLMS, I decided to use TOGAF (**The Open Group Architecture Framework**). I went with TOGAF because it provides a comprehensive and iterative approach to enterprise system development, ensuring alignment between business goals and IT infrastructure.

Unlike simpler (Zachman) frameworks, TOGAF supports **continuous improvement**, making it ideal for a logistics company that must adapt to global operational changes. The TOGAF framework provides a structured approach to enterprise architecture through its Architecture Development Method (The Open Group, 2018).

Application of the TOGAF ADM Cycle

Link of the diagram: https://lucid.app/lucidchart/68b30d0b-5b9d-4f93-84d9-fef1a7b11b43/edit?invitationId=inv_ec276cb0-9f5c-4d94-9eb7-16d9b0132143



Microservices architecture enables independent deployment and scalability of system components (Newman, 2015).

Phase A: Architecture Vision

This phase defines the high-level vision of GLMS as a unified logistics platform. The primary goal is to eliminate data silos and improve operational transparency. Stakeholders include executives, logistics managers, and finance teams.

Phase B: Business Architecture

Key business processes are identified and modeled:

- Contract lifecycle management
- Service request processing
- Financial transaction handling

Phase C: Information Systems Architecture

Data Architecture:

- Centralized database storing clients, contracts, service requests, and invoices
- Ensures data integrity and consistency

Application Architecture:

Microservices-based design:

- Contract Service
- Service Request Service
- Finance Service

Phase D: Technology Architecture

This phase defines the technical infrastructure:

- Backend: Node.js / C# (.NET for enterprise readiness)
- Database: PostgreSQL
- API Communication: RESTful services
- Deployment: Docker Infrastructure

3. Framework Application (Practical)

The TOGAF framework is applied specifically to the logistics context of TechMove:

- Contracts are modeled as core business entities within the Data Architecture
- Service Requests are mapped to application services that enforce validation rules
- Finance processes integrate with external APIs for real-time currency conversion

So Unlike generic implementations, this architecture directly reflects TechMove's operational requirements. The visual architecture (included in diagrams) demonstrates how business processes, data entities, and services interact within a structured enterprise framework.

4. Design Patterns Selection & Justification

To ensure that the system has flexibility, maintainability, and scalability, the three Gang of Four (GoF) design patterns are implemented

1. Factory Pattern

This pattern is used to create different types of service requests ranging from (air freight, road freight) without tightly coupling the system to specific classes

Justification:

This will solve the problem of rigid request creation logic, which will allow the system to easily support new logistics services in the future

2. Observer Pattern

This pattern is used for contract status monitoring. When a contract changes state (e.g., becomes expired), all dependent systems are automatically notified.

Justification

This phase ensures real-time updates and compliance, addressing the issue of missed contract expiration in the legacy system.

3. Strategy Pattern

This strategy pattern is used for currency conversion logic

Justification

It allows the system to switch between

- API-based conversion
- Cached conversion
- Manual conversion

5. Design Patterns Logics & Structure

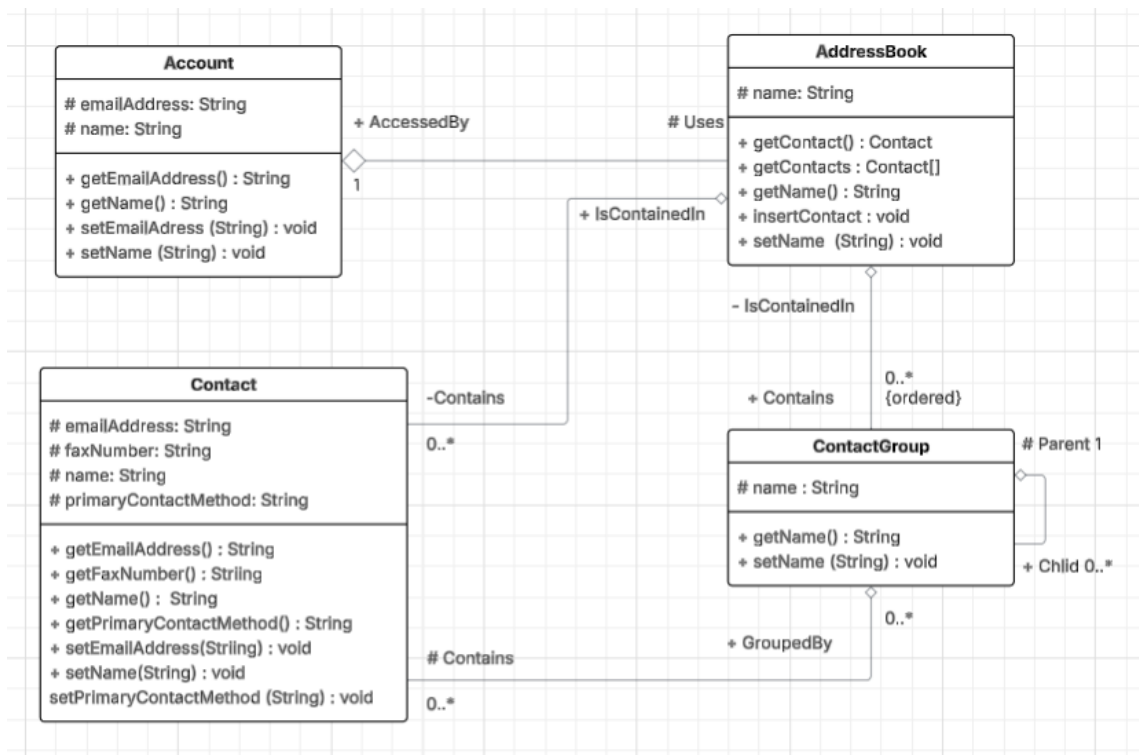
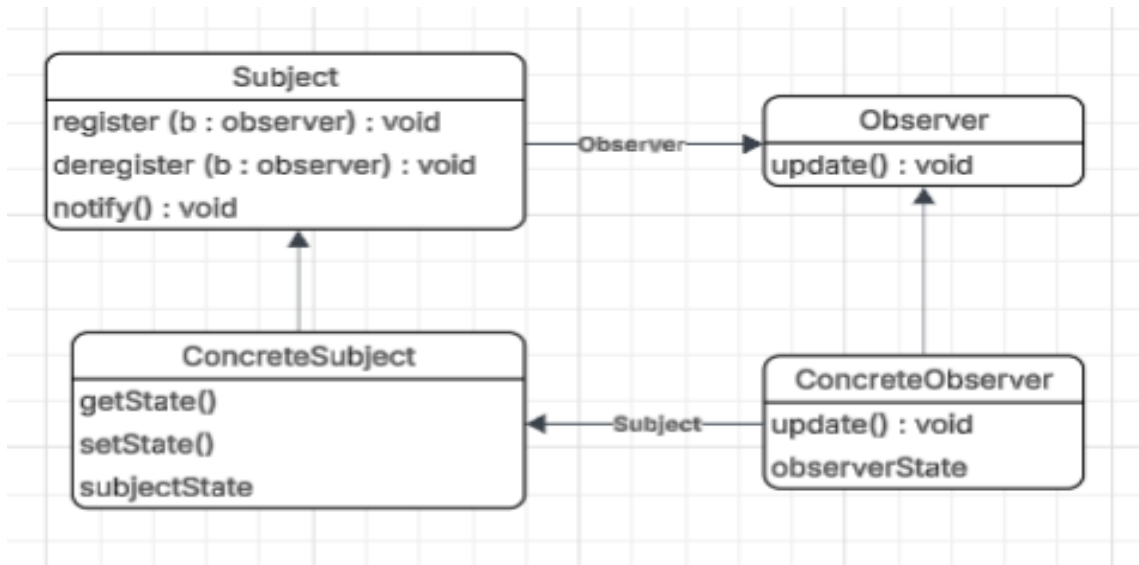
This implementation of these patterns follows a structured and decoupled approach

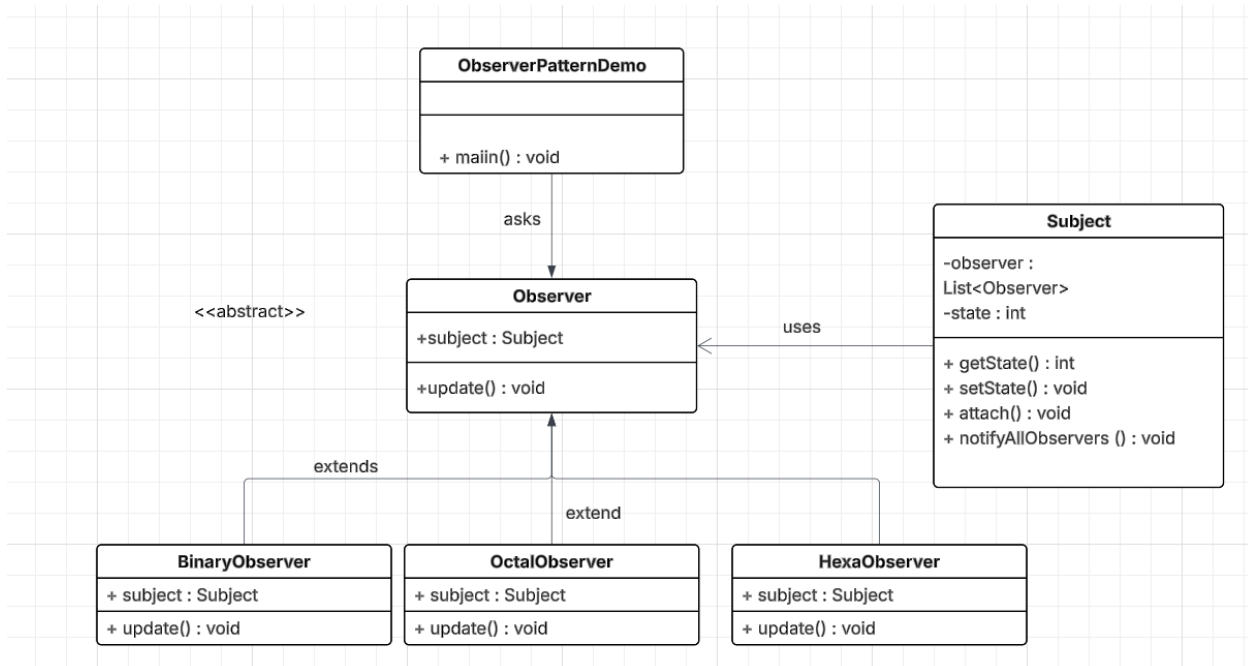
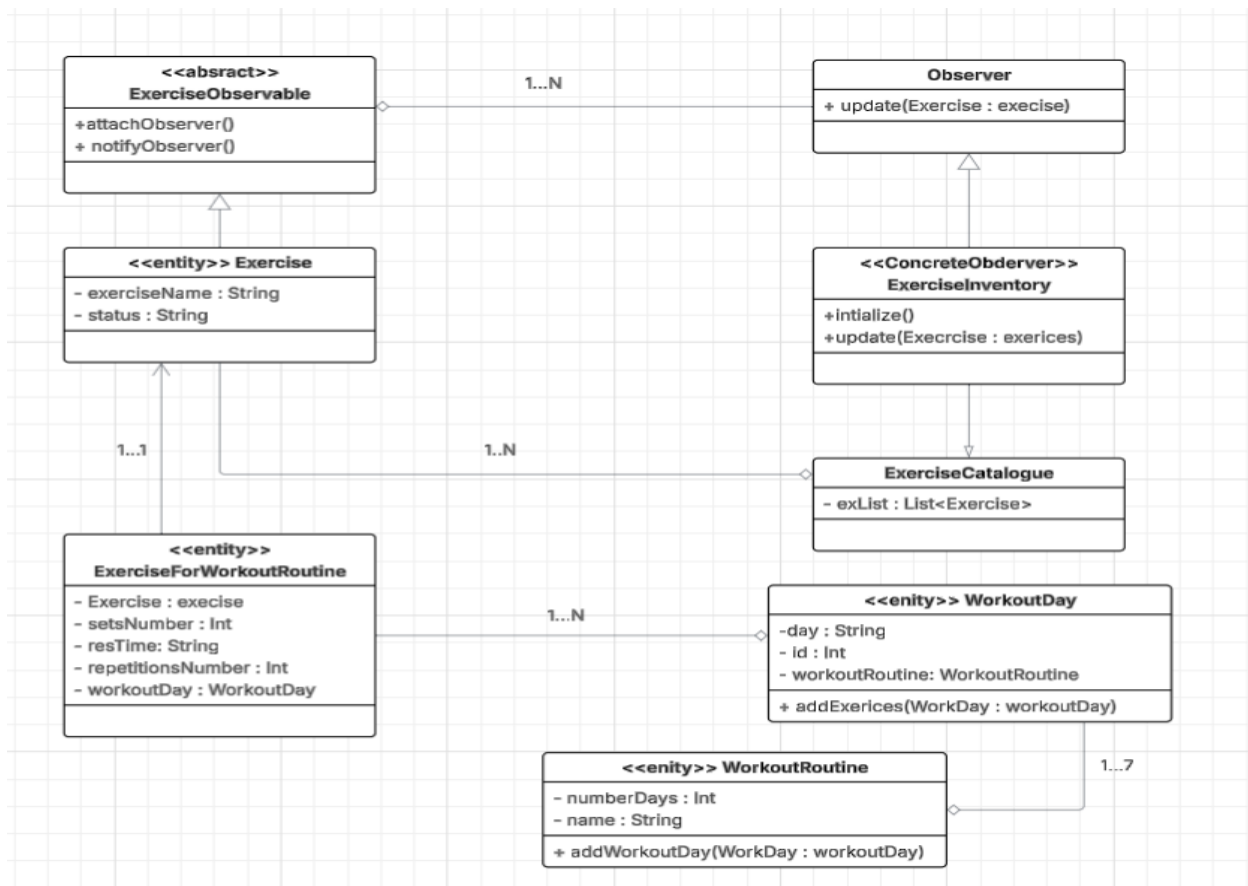
- The Factory Pattern encapsulates object creation logic, ensuring new request types can be added without modifying existing code.
- The Observer Pattern uses event-driven communication, where the Contract class acts as the subject and notifies observers such as **NotificationService** and **FinanceService**.
- The Strategy Pattern uses dependency injection, allowing the **CurrencyContext** to dynamically select the appropriate conversion strategy at runtime.

These patterns are based on established object-oriented design principles that promote flexibility and maintainability (Gamma et al., 1994).

6. UML Modeling

UML Diagram: Observer Design Pattern





The UML Class Diagram illustrates the interaction between the three design patterns using industry-standard notation:

- Interfaces are clearly defined using <<interface>>
- Inheritance and implementation relationships are shown using correct UML arrows
- Composition is used to represent strong relationships (e.g., Contract and Observers)
- Methods and attributes include proper visibility (+ public, - private) and data types

7. Scalability & Non-Functional Requirements Strategy

Availability

The GLMS system is designed to achieve 99.9% uptime, ensuring continuous operation across global logistics processes.

Techniques:

- Load-balanced servers
- Redundant infrastructure
- Failover mechanisms

Interoperability

The system integrates with external and internal systems through:

- RESTful APIs
- JSON data exchange

This allows seamless communication with:

- Currency exchange services
- Third-party logistics systems

Scalability in distributed systems is achieved through strategies such as load balancing, caching, and horizontal scaling (Bass, Clements and Kazman, 2012).

Scalability Strategy

1. Horizontal Scaling

- Multiple instances of each microservice
- Automatically handles increased load

2. Load Balancing

- Distributes traffic evenly across servers
- Prevents bottlenecks

3. Caching (Redis)

- Frequently accessed data (e.g., active contracts, exchange rates) stored in memory
- Reduces database load

In-memory caching technologies such as Redis significantly improve system performance and reduce database load (Redis Ltd., 2023).

4. Database Scaling

- Read replicas for high query performance
- Sharding for large datasets

5. Containerization (Docker)

- Rapid deployment of services
- Consistent environments
- Easy scaling across cloud platforms

8. Conclusion

The GLMS architecture provides a comprehensive solution to TechMove's operational challenges by introducing a modern, scalable, and enterprise-ready system.

Through the use of:

- The TOGAF framework for structured architecture
- GoF design patterns for maintainability
- Microservices and containerization for scalability

The system ensures:

- Improved efficiency
- Enhanced compliance
- High availability
- Future-proof scalability

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley.

The Open Group (2018) TOGAF® Version 9.2. Available at: <https://www.opengroup.org/togaf> (Accessed: 10 April 2026).

Newman, S. (2015) Building Microservices: Designing Fine-Grained Systems. Sebastopol: O'Reilly Media.

Bass, L., Clements, P. and Kazman, R. (2012) Software Architecture in Practice. 3rd edn. Boston: Addison-Wesley.

Fielding, R.T. (2000) Architectural Styles and the Design of Network-Based Software Architectures. Doctoral dissertation. University of California, Irvine.

Redis Ltd. (2023) What is Redis? Available at: <https://redis.io> (Accessed: 10 April 2026).

Docker Inc. (2024) What is Docker? Available at: <https://www.docker.com> (Accessed: 10 April 2026).